

Overflow Labs

Empresa de ciberseguridad

Alumnos: Juan Pablo Figueroa, Leonel Rosso, Paola Balderrama, Marcos Torres

Materia: Testing.

Comision: 331

1. Objetivos

El presente documento detalla la propuesta de servicios de nuestra consultora, enfocada en la mejora de la seguridad en sistemas de bajo nivel, simulando un entorno laboral real para comunicar soluciones técnicas de manera accesible a nuestros clientes

2. Puntos a considerar

2.1. Presentación de la empresa ficticia

- **Nombre de la empresa:** Overflow Labs.
- **Roles de los representantes:**
- **Rubro:** Auditoría de Código Seguro y Ciberseguridad.
- **Misión:** Proteger los activos digitales críticos de nuestros clientes erradicando vulnerabilidades de memoria en software de bajo nivel, garantizando la máxima seguridad operativa.
- **Visión:** Convertirnos en el estándar de confianza técnica para el sector financiero en América Latina frente a las nuevas amenazas informáticas.
- **Tipo de clientes:** Bancos, Fintech y empresas con cajeros automáticos o sistemas transaccionales.

2.2. Problema del cliente

- **Cliente ficticio:** Banco "GlobalBank".
- **El Problema:** El banco sufrió un ataque donde los hackers lograron saltarse la pantalla de login de sus cajeros automáticos.
- **Riesgo:** Pérdida millonaria por robo de efectivo y daño irreparable a la reputación de la entidad por la exposición de la vulnerabilidad.

2.3. Problema del cliente

- **Qué es:** Auditoría preventiva de Desbordamiento de Búfer en código C.
- **Para qué sirve:** Para detectar si un sistema está aceptando más datos de los que su memoria puede almacenar, lo que permite inyectar código malicioso.
- **Qué problemas resuelve:** Evita que el software colapse o deje "puertas traseras" abiertas ante ingresos de texto malintencionados.

2.4. ¿Cómo lo implementarían?

- **Paso 1:** Escaneo del código fuente del cliente con analizadores estáticos para buscar funciones inseguras.
- **Paso 2:** Simulación controlada del ataque en un entorno seguro para verificar la vulnerabilidad.
- **Paso 3:** Reescritura del código defectuoso aplicando prácticas de programación segura.
- **Paso 4:** Nueva prueba de estrés para confirmar que la brecha está cerrada.

2.5. Herramientas

- **Cppcheck:** Un analizador estático que utilizamos para revisar el código fuente en C sin ejecutarlo; sirve para detectar automáticamente el uso de funciones inseguras desde el primer momento.
- **GDB (GNU Debugger):** Una herramienta de depuración que usamos para analizar la memoria en tiempo real y rastrear exactamente en qué momento el programa colapsa durante un ataque.

2.6. Demo / Ejemplo práctico

Mostraremos un programa simple de 10 líneas en C que simula el acceso al cajero.

- **El Error:** El sistema original pide la contraseña usando la función insegura **gets()**. Si el atacante ingresa una contraseña de 100 letras, el programa colapsa y le da acceso al sistema.
- **La Solución:** Mostramos cómo nuestra consultora cambia esa función por la alternativa segura **fgets()**, la cual limita la cantidad de caracteres y bloquea el ataque.

2.7. ¿Por qué elegirlos?

- **Diferencial:** Nos enfocamos exclusivamente en la raíz de los ataques a nivel de memoria, algo que los antivirus tradicionales no pueden frenar.
- **Impacto económico:** Prevenir un ataque cuesta una fracción ínfima en comparación con las multas legales y el robo directo que sufre una institución financiera vulnerada.

2.8. Conclusión

- **Qué aprendimos:** Que la seguridad de un sistema multinacional puede depender enteramente de la correcta elección de una sola función en el lenguaje C.
- **Cuándo usarlo:** Recomendamos este servicio de auditoría antes de lanzar cualquier actualización de software que maneje datos sensibles o interactúe directamente con el público.

3. Roles para Overflow Labs

- **Auditor de Código:** Este rol se encarga de liderar la revisión del código fuente. Su tarea principal es utilizar herramientas de análisis estático, como Cppcheck, para detectar automáticamente el uso de funciones inseguras en C desde el primer momento.
- **Especialista en Pruebas de Penetración (Pentester):** Es la persona responsable de simular los ataques de manera controlada. En el caso práctico, este rol sería el encargado de explicar cómo un atacante ingresa una contraseña excesivamente larga para provocar el colapso del sistema y ganar acceso.
- **Analista de Depuración (Debugger Analyst):** Un perfil técnico enfocado en el análisis forense del fallo. Utiliza herramientas de depuración como GDB para analizar la memoria en tiempo real y rastrear exactamente en qué momento el programa colapsa durante el ataque.
- **Ingeniero de Calidad (QA Engineer):** Es el responsable de validar que la solución implementada sea efectiva y no rompa el programa. Se encarga de documentar y confirmar que el cambio de la función insegura `gets()` por la alternativa segura `fgets()` bloquea exitosamente el ataque.